

LINQ Count Method in C# with Examples

In this article, I will discuss the **LINQ Count Method in C#** with Examples. Please read our previous article discussing the [LINQ Average Method in C#](#) with Examples. The LINQ Count() Method belongs to the category of Aggregate Operators. As part of this article, we will discuss the following pointers.

1. **What is the LINQ Count Method in C#?**
2. **Multiple Examples using both Method and Query Syntax.**
3. **When should you use the LINQ Count Method in C#?**

What is the LINQ Count Method in C#?

The LINQ Count Method returns the number of elements present in the collection or the number of elements that have satisfied a given condition. It's part of the **System.Linq** namespace can be used with arrays, lists, or any type that implements the **IEnumerable<T>** interface. Two overloaded versions of the LINQ Count() Extension method are available. They are as follows.

```
public static int Count<TSource>(this IEnumerable<TSource> source);  
  
public static int Count<TSource>(this IEnumerable<TSource> source, Func<TSource, bool> predicate);
```

As you can see in the above image, the first overloaded version does not take any parameter; it simply returns the count of the number of elements present in the collections. On the other hand, the second overloaded version takes one predicate as a parameter, and it returns the count of the number of elements that have satisfied the given condition. As you can see in the above image, both methods' return types are int. So, it will always return an integer number, which indicates the number of elements present in the collection or satisfies the given condition.

Example to Understand LINQ Count Method in C#:

Let us understand the Count() method in C# with Examples. The following example returns the number of elements in the collection using Query and Method Syntax. Unlike the Sum, Average, Min, and Max Aggregate functions, we can use the Count Method with any data type. In the below example, we are using integer collection. We don't have any operator called count in the LINQ query syntax, so we need to use mixed syntax.

```
using System;  
using System.Linq;  
namespace LINQDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,  
            //Using Method Syntax
```

```

        int MScount = intNumbers.Count();
        //Using Query Syntax
        var QScout = (from num in intNumbers
                      select num).Count();
        Console.WriteLine("No of Elements = " + MScount);
        Console.ReadKey();
    }
}

```

Output: **No of Elements = 10**

Example to Understand Count Method with Where Extension Method in C#

Let us see an example to Understand how we can use the LINQ Count Method and the LINQ Where Extension Method using both Method and Query Syntax. Now, our requirement is to return the number of elements present in the collection which are greater than 40. The following example code exactly does the same.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,
            //Using Method Syntax
            int MScount = intNumbers.Where(num => num > 40).Count();
            //Using Query Syntax
            var QScout = (from num in intNumbers
                          where num > 40
                          select num).Count();
            Console.WriteLine("No of Elements = " + MScount);
            Console.ReadKey();
        }
    }
}

```

Output: **No of Elements = 6**

Example to Understand How to use LINQ Count Method with Predicate in C#

Let us see an example of using the LINQ Count Method with Predicate in C# using both Method and Query Syntax. Instead of using the Where Extension method to filter the data, we can also use the other overloaded version of the Count method, which takes a Predicate as a parameter. Within that predicate, we can write the logic to filter the data. In the example below, within the Count Method, we use a Predicate and provide the condition of whether the number is greater than 40. If the number is greater than 40, we return true;

otherwise, we return false. The following example will return the number of elements present in the collection which are greater than 40.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,

            //Using Method Syntax with a Predicate
            int MSCount = intNumbers.Count(num => {
                if (num > 40)
                    return true;
                else
                    return false;
            });
            //Using Query Syntax with a Predicate
            int QSCount = (from num in intNumbers
                           select num).Count(num => {
                               if (num > 40)
                                   return true;
                               else
                                   return false;
                           });
            Console.WriteLine("No of Elements = " + MSCount);
            Console.ReadKey();
        }
    }
}
```

Output: No of Elements = 6

Note: It can be used with or without a predicate, depending on whether you want to count all elements or only those that meet a condition. If the sequence is empty or no elements satisfy the condition, Count returns 0.

Example to Understand LINQ Count Method with Complex Type in C#:

Let us see an example of how to use the Count Method with Complex Type in C#. We are going to work with the following Employee class. As you can see, the following Employee class contains four properties, i.e., **ID**, **Name**, **Salary**, and **Department**, as well as contains one method, i.e., **GetAllEmployees()**, which will return the list of all the employees and this is going to be our data source.

```
using System.Collections.Generic;
namespace LINQDemo
{
```

```

public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public int Salary { get; set; }
    public string Department { get; set; }
    public static List<Employee> GetAllEmployees()
    {
        List<Employee> listStudents = new List<Employee>()
        {
            new Employee{ID= 101,Name = "Preety", Salary = 10000, Department = "Sales"},
            new Employee{ID= 102,Name = "Priyanka", Salary = 15000, Department = "Sales"},
            new Employee{ID= 103,Name = "James", Salary = 50000, Department = "IT"},
            new Employee{ID= 104,Name = "Hina", Salary = 20000, Department = "Sales"},
            new Employee{ID= 105,Name = "Anurag", Salary = 30000, Department = "Sales"},
            new Employee{ID= 106,Name = "Sara", Salary = 25000, Department = "IT"},
            new Employee{ID= 107,Name = "Pranaya", Salary = 35000, Department = "Sales"},
            new Employee{ID= 108,Name = "Manoj", Salary = 11000, Department = "Sales"},
            new Employee{ID= 109,Name = "Sam", Salary = 45000, Department = "Sales"},
            new Employee{ID= 110,Name = "Saurav", Salary = 25000, Department = "Sales"}
        };
        return listStudents;
    }
}

```

Now, our requirement is to calculate the Number of Employees present in the collection. The following example returns the Number of Employees present in the collection using the LINQ Count Method.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSCount = Employee.GetAllEmployees().Count();
            //Using Query Syntax
            var QSCount = (from emp in Employee.GetAllEmployees()
                           select emp).Count();
            Console.WriteLine("Total No of Employees = " + QSCount);
            Console.ReadKey();
        }
    }
}

```

```
}
```

Output: **Total No of Employees = 10**

Example using Count and Where Extension Method in C#:

Let us see an example of Count and Where Extension Methods with Complex Type using Method and Query Syntax. Our requirement is to return the number of employees who belong to the IT department. The following example exactly does the same. Using the Where Extension Method, we filter the IT department employees and then call the Count method, which will return the number of employees in the IT department.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSCount = Employee.GetAllEmployees()
                                .Where(emp => emp.Department == "IT")
                                .Count();

            //Using Query Syntax
            var QSCount = (from emp in Employee.GetAllEmployees()
                           where emp.Department == "IT"
                           select emp).Count();

            Console.WriteLine("Total No of Employees of IT Department = " + QSCount);
            Console.ReadKey();
        }
    }
}
```

Output: **Total No of Employees of IT Department = 5**

Let's rewrite the previous example using the custom predicate. As you can see, within the Count method, we are checking the Department property value, and if the Department is It, we are returning true; otherwise, we are returning false.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSCount = Employee.GetAllEmployees()
                                .Count(emp => {
```

```

        if (emp.Department == "IT")
            return true;
        else
            return false;
    });
//Using Query Syntax
var QSCount = (from emp in Employee.GetAllEmployees()
               select emp).Count(emp => {
                if (emp.Department == "IT")
                    return true;
                else
                    return false;
            });
Console.WriteLine("Total No of Employees of IT Department = " + QSCount);
Console.ReadKey();
    }
}
}

```

Output: Total No of Employees of IT Department = 5

When should you use the LINQ Count Method in C#?

Here are some common situations when you should use the Count method:

- **Counting Items in a Collection:** The most straightforward use of Count is to determine the total number of elements in a collection, such as an array or a list.
- **Conditional Counting:** Count can be used with a lambda expression to count the number of elements that satisfy a specific condition. For example, counting the number of items in a list that are greater than a certain value.
- **Database Queries:** In LINQ to SQL or LINQ to Entities, Count counts the number of rows in a database table meeting certain criteria. This is particularly useful for generating statistics or performing data validation.
- **Grouped Data:** When used in conjunction with GroupBy, Count can determine the number of elements in each group, allowing for more complex data aggregations and analysis.
- **Empty Collection Check:** Count can be used to check if a collection is empty (i.e., its count is zero). However, using Any() is often more efficient for this specific use case, especially for large collections.
- **Count with Join Operations:** Count can be used to find the number of matching elements in different collections in more complex LINQ queries involving join operations.
- **Performance Considerations:** While Count is generally efficient, its performance can depend on the type of collection it's applied to. For instance, counting elements in a List<T> is very fast, but in a LinkedList<T>, it might require iterating through all elements.
- **Limitations:** Be aware that for very large collections, the result of Count can exceed the maximum value of an int. In such cases, consider using LongCount instead, which returns a long.

In the next article, I will discuss the [LINQ Aggregate Method in C#](#) with Examples. I hope you understand the use and need of the [LINQ Count Method in C#](#) with Examples. I hope you enjoy this LINQ Count Method in C# with Examples article.